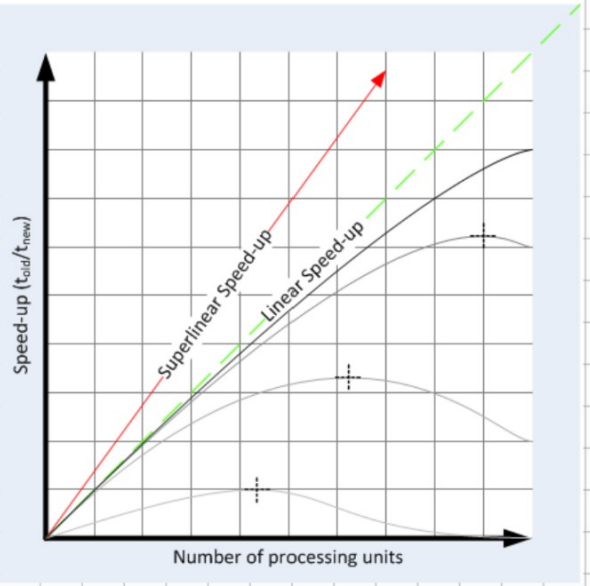
LECTURE 1

MOORE'S LAW

N° of transistor doubles every 18-24 months

SPEED UP REGIME



speed up $a = \frac{t_{old}}{t_{new}} = \frac{t_{ser}(M)}{t_{par}(M,N)}$

$1 < a(N) < N$

linear speed up $a(N) = N$

superlinear speed up $a(N) > N$ (usually not possible)

efficiency $e = \frac{t_{ser}(M)}{N \cdot t_{par}(M,N)}$ $(0 < e < 1)$

AMDAHL'S LAW

Model to find the maximum improvement in terms of performance

- Only a fraction of time can be parallelized (parallel fraction p)
- The other fraction is the serial one: (serial fraction s)

→ $s + p = 1$

- The fraction p is processed by N parallel execution units

$$a = \frac{1}{s + \frac{p}{N}} = \frac{1}{(1-p) + \frac{p}{N}}$$

→ The speed limit is related to s, not to N

PERFORMANCE SCALING

CLASSIC DENNARD SCALING

↓ As transistor shrank they could switch faster without using more power per unit area

$$\text{Perf.}\left(\frac{ops}{s}\right) = \underbrace{\frac{\text{instructions}}{\text{cycle}}}_{\propto \text{PipelineCount} \cdot \text{PipelineDepth}} \cdot \underbrace{\text{frequency}}_{\text{↳ Scales with features}}$$

Power → we eventually hit a physical limit known as the power wall

- the equation: $P = \alpha f C V^2 + V I_{leakage} \propto f^3$

  P: Total power

  α: Activity factor, the part of chip that actually commutes during a clock cycle.

- f: Clock frequency
- C: Load capacity
- V: Voltage
- $VI_{leakage}$: Static (idle) power consumption,
- $\propto f^3$: the cubic proportionality to frequency. Dinamic power is given by $\alpha f C V^2$

  ↓

  "is proportional to"

The power consuption is proportioned with the cube of frequency

⇓

Higher frequencies lead to high power drain.

NEW WAY → POST-DENNARD SCALING

We can't boost the frequence after the power wall is reached and in order to improve the performance we have to cut cost:

Operator cost → energy needed for single operation

Data movement cost → energy used to move the information from memory to CPU and viceversa.

TRANSITION TO PARALLEL ARCHITECTURES

From single core at high frequency to high number of, smaller cores with lower frequencies

AMDAHL'S LAW

Calculates the maximum speedup in latency of the execution of a task when only a specific fraction of the system is improved, most commonly applied to parallel computing.

VARIABLES:

- T_{old}: the initial execution time
- f: the fraction that can be improved → (Part of code that can be parallelized)
- $(1-f)$: the fraction that cannot be improved
- S: Speedup factor applied to improvable portion
- T_{new}: The execution time after improvements

CALCULATE NEW EXECUTION TIME

$$T_{new} = \underbrace{(1-f)}_{\text{Sequential part (Unchanged)}} + \underbrace{\frac{f}{S}}_{\text{Parallelizable part of code}}$$

CALCULATE OVERALL SPEEDUP

Total speedup is defined as the ratio of the original execution time:

$$\text{Speedup overall} = \frac{T_{old}}{T_{new}}$$

by substituting the normalized initial time ($T_{old}=1$) and the equation for T_{new} we arrive at:

$$\text{Amdahl's law}: \frac{1}{(1-f)+\frac{f}{S}}$$

$$\text{maxSpeedUP} \longrightarrow : \frac{1}{(1-f)} \quad (\text{if } S(n \text{ procesor}) \to \infty)$$

CORRECTNESS OF THIS LAW

Is a simplified upper bound on how much you can parallelize/speed-up the code.

REPORTING TOO HIGH PERFORMANCE:

Amdahl's law assumes splitting work across processors is free.
it ignores the physical realities of parallel computing such as synchronization locks thread creation, load inbalance. Ignoring this things lead to predict a maximum speedup that is too high

EXAMPLE: Amdahl's law might predict 900x based on the parallel math equations. However the machine spend a lot of time communicating over the network to share intermediate data.

↳ Due to communication bottleneck the actual speedup is 100X

REPORTING TOO LOW PERFORMANCE

Amdahl's law can predict a speedup that is too low because it fails to account for hardware architectural benefits (memory hyerarchy) and how humans utilize large computing systems. (Gustafson's law)

EXAMPLE: in real applications like weather forecasting, upgrading from 10 to 1000 processors is rarely used to solve small problems faster (which Amdhal's law models). → The problem size is not fixed anymore.

Instead, users increase the grid resolution of simulations (Weak scaling)

↳ How much bigger is the problem i can solve in the same time with the processors improvement.

AMDAHL'S CAP: in the original scenario (1 hour sequential setup and 9 hours parallel), the sequential part is (1-f) is 10% of the time.
Amdahl's formula stricty dictates that the maximum theoretical speedup is capped at $10x \left(\frac{1}{0,10}\right)$ even with billions of processors.

EXERCIZE:

CASE A: $\begin{cases} P = 0.5 \\ N = 4 \end{cases}$ LAW: $\frac{1}{(1-P)\cdot\frac{P}{N}} = \frac{1}{0,5\cdot\frac{0,5}{4}} = \frac{1}{0,0625} = 16$

CASE B: $\begin{cases} P = 0.2 \\ N = 8 \end{cases}$ LAW: $\frac{1}{(1-P)\cdot\frac{P}{N}} = \frac{1}{0,8\cdot\frac{0,2}{8}} = \frac{1}{0,08} = 12,5$

CONCLUSION: the case A has higher "parallelizable portion of code" and even with the lower amount of processor achieve a higher speed up factor.

on the other hand the case B has higher n° of processor but the lower percentage of parallelizable software makes the speedup factor smaller

We can conclude that is more important to have high P rather than a higher num of processors.

LECTURE 2

comparison between Die shots of CPU and GPU

↳ if process data can fit into CPU cache latency will be low otherwise ram accesses are required

CPU vs GPU

GPU are latency tolerant → fill waiting time with other operations

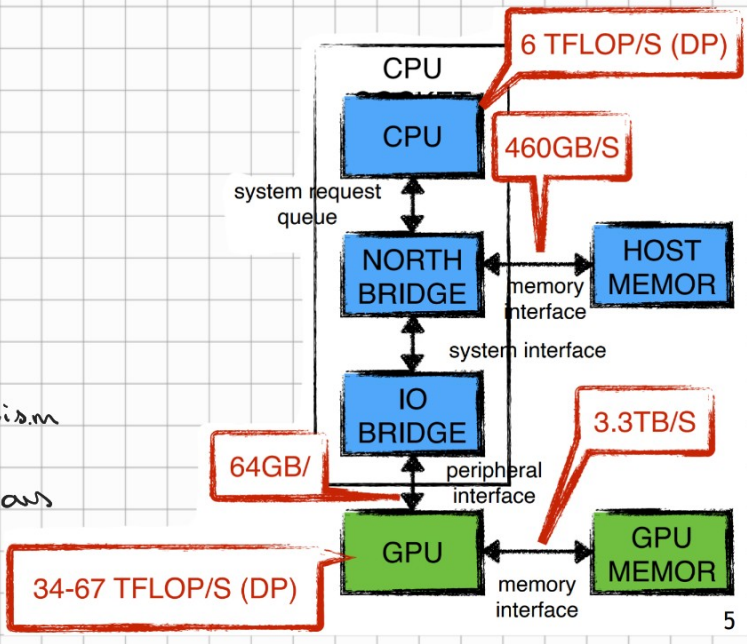
CPU is looking for latency minimization

CUDA

OVERVIEW ARCHITECTURE

NVIDIA CUDA

- Compute Kernel as C program
- explicit data and thread parallelism
- Computing, not graphic operations
- Host communication



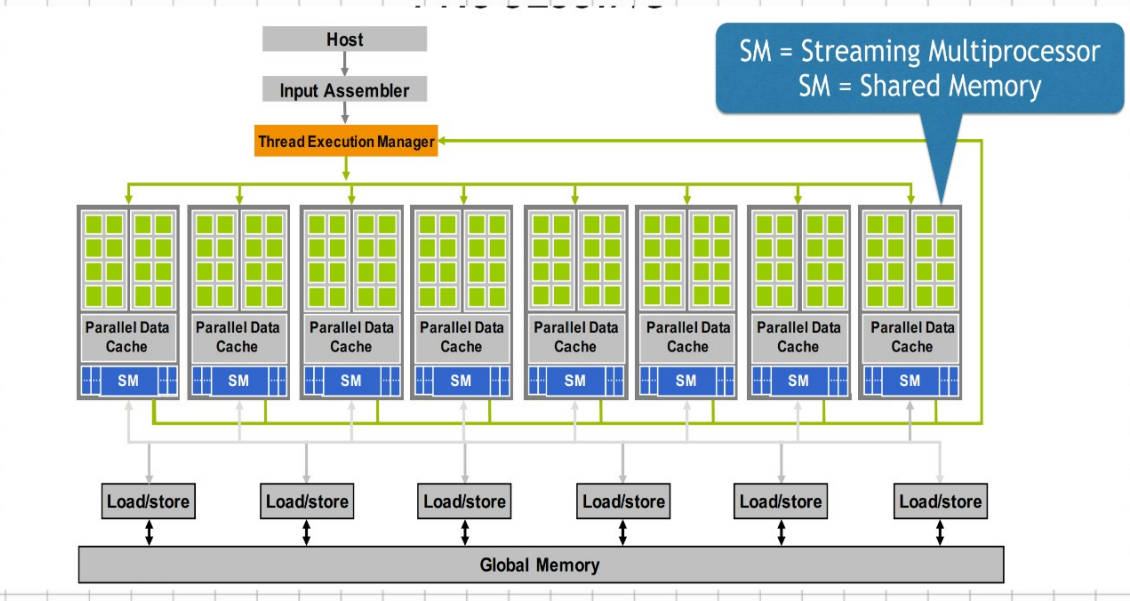
MEM HIERARCHY

- Host memory
- GPU (device) memory
- GPU on chip memory

LIMITATION:

- Pointers are not so good
- Load/store architecture
- Different synchronizations need to be compatible

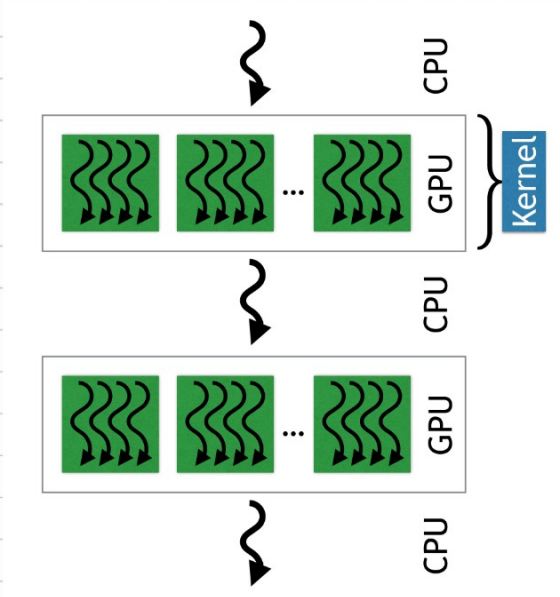
G80 - ARCHITECTURE



CUDA PROGRAMMING MODEL

CPU part of program → no parallelism

GPU part of program → high parallelism



Current execution

Non blocking thread execution

explicit synchronization

C extension with tree main extensions

Hierarchy of thread

Shared memory

Barrier synchronization

exploiting parallelism

KERNEL LAUNCH

Kernel → N-fold execution by N threads

↳ executed on GPU after being called by CPU

Configuration → how many threads and blocks

thread block → has up to 3 dimension (1024 x 1024 x 64)

↓

Can be present different blocks (limited to (64K - 1) for each dimension)

↓

independently executed and not in order

THREAD HIERARCHY

Grid of thread blocks of equal size

→ Recommended n° of blocks = 2x number of SMs

THREAD COMMUNICATION

→ Communication within one thread

- shared memory
- Atomic operation
- Barrier synchronization

THREAD OF DIFF BLOCK CANNOT INTERACT → except for global memory

Memory hierarchy

GLOBAL MEMORY

- Communication between host and device
- Accessible from all thread
- High latency
- Sensitive to fine grained accesses

SHARED Small - fast - On Chip memory

- has the same lifetime of the thread
- Organized in banks (16-32 banks with 32 bit width
- low order interleaving